Day 73: Dot-Product (Luong) Attention, and a Real Comparison With Additive

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 4 | Complete study guide

Watch: Bahdanau vs Luong Attention — Additive vs Multiplicative (SourceCode AI) | Bahdanau Attention Vs Luong Attention (CampusX)

Script: `day73_dot_product_attention.py` · every number in this guide is copied by program from `day73_run_output.txt`, the real output of that script.

Day 71's attention scored each encoder state with an **additive** score. Luong et al. (2015) proposed **multiplicative** scores: the plain **dot** product s · h, and the **general** score (s W) · h. Here they are compared like for like on Day 70's hardest test, with everything except the score kept identical. At hidden 32 all three learn 20-digit copies, at different speeds: the general score fastest (at 20 digits 0.806 after 2,500 steps, against 0.000 additive and 0.018 dot), the additive score the steadiest, and the dot score, which has no weights at all, the slowest and the cheapest. With a smaller state (hidden 16) the dot score fails: 0.000 at 20 digits, where the other two stay near 1.

0. Where today fits

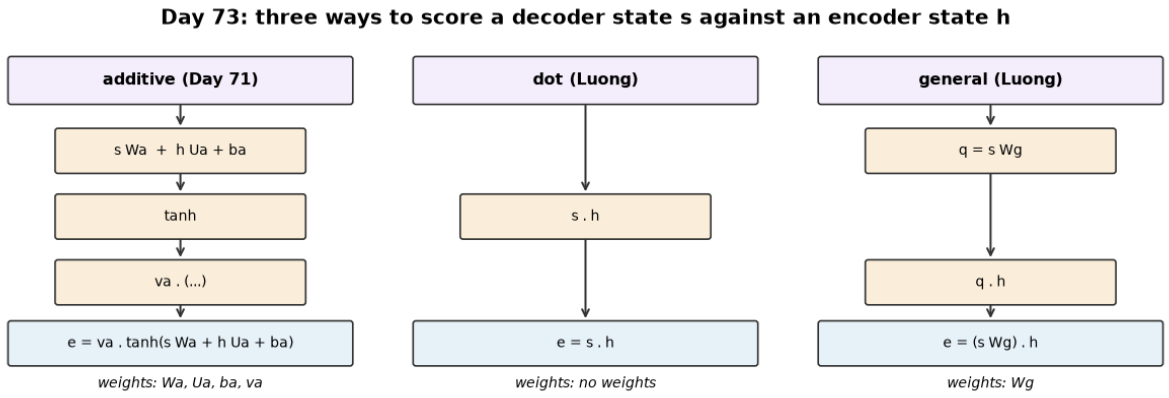
DEFINITION — start here, no prior knowledge assumed

An attention **score** measures how well an encoder state h matches the decoder's state s. **Additive** (Day 71): e = v · tanh(s W_a + h U_a + b). **Dot**: e = s · h, no weights. **General**: e = (s W_g) · h, one weight matrix. The dot and general scores are called **multiplicative** because s and h are multiplied directly.

REAL-WORLD SCENARIO

Three ways to judge whether a CV matches a job: a trained recruiter scoring with a checklist (additive), counting the words the two share (dot), or translating the job description into CV language first and then counting (general).

SYSTEM DIAGRAM



The three scores. Everything else in the model is Day 71's.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

READ THIS FIRST

What is and is not Luong's model here. Luong et al. also changed other parts of the decoder (for example, using the *current* decoder state as the query and an extra layer after attention). Those are not made today, so that the score is the only difference from Day 71.

1. Three scores for the same step

DEFINITION — start here, no prior knowledge assumed

Day 71's hand example scored three ways. With W_g = identity the general score is exactly the dot score; another W_g changes which states it prefers.

REAL-WORLD SCENARIO

Scoring the same three CVs with each of the three methods.

MATHEMATICS

$$\text{dot: } e_i = s \cdot h_i; \quad \text{general: } e_i = (s\,W_g) \cdot h_i; \quad \text{additive: } e_i = v_a \cdot \tanh(s\,W_a + h_i\,U_a + b_a)$$

$$\text{weights } \alpha = \text{softmax}(e), \quad \text{context} = \Sigma_i\, \alpha_i\, h_i \quad \text{(Day 71, unchanged)}$$

SPECIFICATION

Setup	h = (1, 0), (0, 1), (1, 1); s = (1, –1); additive with Day 71's weights; general with W_g = I, then [[2, 0], [0, 0]].

Why this test

The new scores are one line each; checking them by hand makes their behaviour concrete.

IMPLEMENTATION

```
def dot_scores(s_prev, Hs):
    return np.einsum("mh,mnh->mn", s_prev, Hs)
def dot_scores_backward(de, s_prev, Hs):
    ds = np.einsum("mn,mnh->mh", de, Hs)
    dHs = de[:, :, None] * s_prev[:, None, :]
    return ds, dHs
def general_scores(s_prev, Hs, Wg):
    q = s_prev @ Wg
    return np.einsum("mh,mnh->mn", q, Hs), q
def general_scores_backward(de, s_prev, Hs, q, Wg):
    dq = np.einsum("mn,mnh->mh", de, Hs)
    dHs = de[:, :, None] * q[:, None, :]
    return dq @ Wg.T, dHs, s_prev.T @ dq
```

ACTUAL OUTPUT

```
encoder state     dot by hand     dot  general, I  general, Wg  additive
[1.0, 0.0]             1.0000  1.0000      1.0000       2.0000    1.6667
[0.0, 1.0]            -1.0000 -1.0000     -1.0000       0.0000    1.2237
[1.0, 1.0]             0.0000  0.0000      0.0000       2.0000    1.3673
weights                             dot           general, Wg              additive
[1.0, 0.0]                       0.6652                0.4683                0.4196
[0.0, 1.0]                       0.0900                0.0634                0.2694
[1.0, 1.0]                       0.2447                0.4683                0.3110
Actual output from this lesson's run.
```

What it means

The dot score matches the hand values and the general score with W_g = I equals it. The dot score puts 0.6652 of the weight on (1, 0), the state most aligned with s; the general score with [[2, 0], [0, 0]] only looks at the first number, so (1, 0) and (1, 1) tie at 0.4683. The additive score with Day 71's hand-chosen weights is the least decisive here (0.4196 at most).

WHY CHOOSE THIS?

Multiplicative scores are one product: simple to write and fast to compute.

WHY NOT THIS? (TRADEOFFS)

The dot score cannot learn anything about how to compare: it has no weights.

2. Gradient check

DEFINITION — start here, no prior knowledge assumed

The new scores' backward passes, checked through the whole model with Day 71's tiny setup.

REAL-WORLD SCENARIO

Testing each new component on the bench before the full system run.

MATHEMATICS

$$dot:\ ds = \Sigma_i\, de_i\, h_i,\quad dh_i \mathrel{+}= de_i\, s$$

$$general:\ q = s\, W_g;\quad dq = \Sigma_i\, de_i\, h_i;\quad ds = dq\, W_g^{T},\quad dW_g = s^{T}\, dq,\quad dh_i \mathrel{+}= de_i\, q$$

SPECIFICATION

Data	Hidden 4, 3 REVERSE sequences of 5 digits, every entry of every parameter; Day 62's pass rule.

Why this test

In the dot and general models the encoder states are the keys, so the memory now gets gradient through the scores as well as through the weighted sum.

IMPLEMENTATION

```
if self.score == "dot":
    dsp, dH_score = dot_scores_backward(de, st["s_prev"], Hs)
else:
    dsp, dH_score, dWg = general_scores_backward(de, st["s_prev"], Hs, st["t"], P["Wg"])
    g["Wg"] += dWg
dHs += dH_score
```

ACTUAL OUTPUT

```
score     parameter  entries  max relative error  max |difference|   pass
dot              We      224            5.90e-06          5.11e-11   True
dot              be       16            1.05e-07          2.36e-11   True
dot              Wd      288            3.93e-06          3.90e-11   True
dot              bd       16            1.39e-07          2.15e-11   True
dot             Why       80            2.65e-08          3.90e-11   True
dot              by       10            1.14e-08          3.14e-11   True
general          We      224            4.80e-05          3.41e-11   True
general          be       16            1.81e-07          2.78e-11   True
general          Wd      288            1.09e-05          4.04e-11   True
general          bd       16            2.14e-07          3.04e-11   True
general          Wg       16            1.43e-06          2.73e-11   True
general         Why       80            4.94e-08          3.26e-11   True
general          by       10            2.04e-08          2.83e-11   True
every entry passes: dot True, general True
Actual output from this lesson's run.
```

What it means

Every entry passes for both new scores. The additive model is Day 71's own code, checked there.

WHY CHOOSE THIS?

Shorter backward passes than the additive score's.

WHY NOT THIS? (TRADEOFFS)

A tiny model; the trained one runs the same code.

3. The comparison: COPY, 3–20 digits

DEFINITION — start here, no prior knowledge assumed

Day 71's Section 3 exactly: the same data stream, seeds, learning rate and test sequences; only the score differs. The additive rows are therefore Day 71's run repeated.

REAL-WORLD SCENARIO

Three students, the same lessons and the same exam, each with a different note-taking method.

SPECIFICATION

Runs	Hidden 32, 20,000 steps, lr 0.5, clip 1.0, three seeds per score; exact match every 2,500 steps.
Check	The additive curve must reproduce Day 71's: 0.572 at 12 digits after 2,500 steps.

Why this test

It is the like-for-like comparison: anything that differs comes from the score.

IMPLEMENTATION

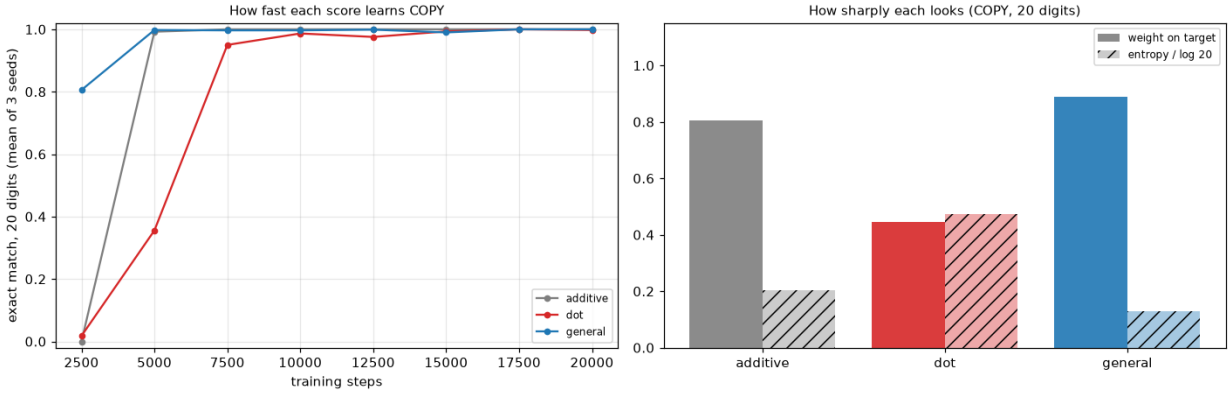
```
def scores(self, s, Hs, keys):
    if self.score == "additive":
        return additive_scores(s, keys, self.P["Wa"], self.P["va"])
    if self.score == "dot":
        return dot_scores(s, Hs), None
    return general_scores(s, Hs, self.P["Wg"])
```

ACTUAL OUTPUT

```
  steps  additive n=12  additive n=20       dot n=12       dot n=20   general n=12   general n=20
   2500          0.572          0.000          0.338          0.018          0.994          0.806
   5000          1.000          0.992          0.824          0.356          0.997          0.998
   7500          1.000          1.000          0.998          0.950          1.000          0.997
  10000          1.000          1.000          1.000          0.987          1.000          0.997
  12500          1.000          1.000          0.998          0.976          1.000          0.999
  15000          1.000          1.000          1.000          0.993          0.996          0.990
  17500          1.000          1.000          1.000          1.000          1.000          1.000
  20000          1.000          1.000          1.000          0.998          1.000          1.000
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
final exact match at 20 digits per seed, and the first checkpoint where every seed is at 1.000 on
    all lengths:
  additive seed 0 1.000  seed 1 1.000  seed 2 1.000   all perfect from: 7500 steps
  dot      seed 0 0.997  seed 1 1.000  seed 2 0.997   all perfect from: not within the budget
  general  seed 0 1.000  seed 1 1.000  seed 2 1.000   all perfect from: 17500 steps
Actual output from this lesson's run.
```



Left: exact match on 20-digit copies during training, mean of three seeds. Right: how sharply each trained model looks (weight on the right input digit, and entropy relative to log 20).

What it means

The additive rows reproduce Day 71 exactly (0.572 and 0.000 at 2,500 steps), so the comparison runs on the same footing.

General is fastest: already 0.806 at 20 digits after 2,500 steps, when additive is at 0 and dot at 0.018. **Additive is steadiest:** every seed is perfect on every length from 7,500 steps on, while general dips and stays perfect only from 17,500. **Dot is slowest** (0.356 at 5,000 steps) and does not stay perfect within the budget, though its final seeds are within a few test sequences of it (0.997).

WHY CHOOSE THIS?

All three solve the task; the choice is about speed, steadiness and cost.

WHY NOT THIS? (TRADEOFFS)

One task and one budget; the ranking may change with the task and with tuning.

4. REVERSE, and a smaller state

DEFINITION — start here, no prior knowledge assumed

The same three scores on REVERSE at hidden 32, and on COPY at hidden 16, where the dot score's query and keys have only 16 numbers.

REAL-WORLD SCENARIO

The same three note-taking methods on a different exam, and with a smaller notebook.

SPECIFICATION

Runs	Three seeds each; final exact match per length.

Why this test

A result that holds on one task and one size may not hold on others.

ACTUAL OUTPUT

```
model                          n=4     n=8    n=12    n=16    n=20  weights
reverse, h 32, additive      1.000   1.000   1.000   1.000   1.000    17866
reverse, h 32, dot           1.000   1.000   1.000   0.999   0.984    15754
reverse, h 32, general       1.000   1.000   1.000   1.000   1.000    16778
copy, h 16, additive         1.000   1.000   1.000   1.000   1.000     5898
copy, h 16, dot              0.959   0.536   0.026   0.000   0.000     4810
copy, h 16, general          1.000   1.000   1.000   1.000   0.997     5066
Actual output from this lesson's run.
```

What it means

REVERSE at hidden 32 repeats Section 3's picture: exact match at 20 digits is 1.000 (additive), 0.984 (dot) and 1.000 (general).

COPY at hidden 16 separates them. Additive 1.000 and general 0.997 at 20 digits, but the dot score fails: 0.959 at 4 digits, 0.536 at 8, 0.026 at 12 and 0.000 at 20. The general model has only 5066 weights against the dot model's 4810: its 16 × 16 matrix is the whole difference.

A likely reason, not tested here. With no weights, the dot score can only make a sharp choice if the decoder state and the right encoder state are themselves large and aligned. LSTM states lie between –1 and 1, and the same 16 numbers must also carry the digits and the decoder's own progress. W_g separates the two jobs: it can stretch and turn the query freely. Whether more training steps would let the dot model get there is also not tested: the budget is the same 20,000 steps for every model.

WHY CHOOSE THIS?

Shows where the weightless score stops being enough.

WHY NOT THIS? (TRADEOFFS)

Three seeds per cell; one smaller size and one task.

5. Where each score looks

DEFINITION — start here, no prior knowledge assumed

Day 72's three measurements for each trained COPY model at 20 digits: argmax on target, weight on target, and entropy (log 20 = 3.00).

REAL-WORLD SCENARIO

Checking whose notes point to the right page, and how precisely.

ACTUAL OUTPUT

```
score       argmax on target  weight on target  entropy
additive               1.000             0.806    0.605
dot                    0.840             0.444    1.417
general                1.000             0.889    0.388
Actual output from this lesson's run.
```

What it means

Weight on the right input digit: additive 0.806, dot 0.444, general 0.889; entropy 0.605, 1.417 and 0.388. The additive and general models put their largest weight on the right digit every time (1.000); the dot model only for 0.840 of outputs, and spreads its weight much more.

Yet the dot models copy 20 digits almost perfectly (Section 3). How they still write the right digit with a more diffuse look is not tested here; it does show that an alignment heatmap and the accuracy are two different measurements.

WHY CHOOSE THIS?

The same alignment measurements as Day 72, so the scores can be compared on how they attend.

WHY NOT THIS? (TRADEOFFS)

Sharpness is not accuracy: all three copy correctly at hidden 32.

6. How large are the scores?

DEFINITION — start here, no prior knowledge assumed

The spread of the raw scores over the 20 encoder positions at the first decoder step, and the largest weight, before and after training.

REAL-WORLD SCENARIO

How far apart the judges' marks are: close marks mean an undecided panel, far-apart marks a decided one.

ACTUAL OUTPUT

```
score       spread, untrained  max weight, untrained  spread, trained  max weight, trained
additive                0.041                 0.054            6.172                0.977
dot                     0.059                 0.055            2.463                0.638
general                 0.029                 0.053            9.451                0.983
Actual output from this lesson's run.
```

What it means

Untrained, all three are small and the weights nearly even (largest weight about 0.055, against 1/20 = 0.05). Training spreads them: general 9.451, additive 6.172, and dot only 2.463, with a largest weight of 0.638 against 0.983 and 0.977. The scores with weights learned to make large, decisive scores; the dot score's can only grow as far as the states themselves allow, which fits Sections 4 and 5.

The opposite problem appears in large models. There the query and key are long vectors whose numbers are not bounded between –1 and 1, and their dot product grows with the length, until the softmax becomes nearly one-hot. Day 75 measures that, and the fix: dividing by √d.

WHY CHOOSE THIS?

Shows how much each score can sharpen, before and after training.

WHY NOT THIS? (TRADEOFFS)

Measured at one step of one task, seed 0.

7. What each score costs

DEFINITION — start here, no prior knowledge assumed

One training step's time for each model, in this run, and the number of weights each score adds.

REAL-WORLD SCENARIO

The recruiter costs a salary; counting shared words costs nothing.

ACTUAL OUTPUT

```
score       ms per step  vs additive  score weights
additive           11.5         1.00           2112
dot                 6.1         0.53              0
general             7.5         0.65           1024
Actual output from this lesson's run.
```

What it means

The dot score adds no weights and has the cheapest step (0.53 times the additive step); the general score adds 1024 weights (0.65 times); the additive score adds 2112. This machine's speed varies between runs, so only these ratios from one run are meaningful.

WHY CHOOSE THIS?

Multiplicative scores are cheaper per step.

WHY NOT THIS? (TRADEOFFS)

The saving matters most when there are many encoder states.

Interview preparation — technical questions

Q: Write the dot, general and additive scores.

A: dot: s . h; general: (s W) . h; additive: v . tanh(s Wa + h Ua + b).

Q: When does the general score reduce to the dot score?

A: When W is the identity matrix.

Q: What does the dot score require of the query and key sizes?

A: They must be equal; the general score's W can map between different sizes.

Q: Which score learned COPY fastest here?

A: General: 0.806 exact match at 20 digits after 2,500 steps.

Q: Why does the dot score need scaling in large models?

A: With long, unbounded vectors its value grows with the length and saturates the softmax (Day 75); in this small model its trained scores were the least spread (2.463).

Q: Which score failed at hidden 16?

A: The dot score: 0.000 exact match at 20 digits, where additive and general stayed near 1.

Interview preparation — theoretical questions

Q: Additive or multiplicative?

A: Multiplicative is cheaper and is what Transformers use (with scaling); additive was steadier here; at hidden 32 all three solved the task.

Q: Why did the dot score learn more slowly, and fail with a small state?

A: It cannot learn how to compare: the encoder and decoder states themselves must line up, using the same numbers that carry the digits. A likely reason, not tested here.

Q: Why keep everything else fixed?

A: So that any difference can only come from the score.

Q: What did Luong change besides the score?

A: The query state and an extra combination layer; not used here, to isolate the score.

Key takeaway and what's next

> **KEY TAKEAWAY**
> With everything else identical and hidden 32, all three scores learn to copy 20 digits. The general score, one weight matrix between query and key, learned fastest; the additive score was the steadiest; the dot score, with no weights at all, was the slowest and the cheapest, and with a 16-number state it failed. The Transformer keeps the dot product but gives it learned projections of query and key (Day 74), much as the general score does, and scales it (Day 75).

- dot: s . h; general: (s Wg) . h; additive: va . tanh(s Wa + h Ua + ba).
- Additive reproduces Day 71 exactly; the comparison is like for like.
- 20 digits after 2,500 steps: general 0.806, dot 0.018, additive 0.000.
- All perfect on every length from: additive 7,500 steps, general 17,500, dot not within 20,000.
- Hidden 16: additive 1.000, general 0.997, dot 0.000 at 20 digits.
- Trained score spread: general 9.451, additive 6.172, dot 2.463.

Suggested watch order, if starting from zero

- 1. Bahdanau vs Luong Attention — Additive vs Multiplicative (SourceCode AI) — the two families of scores side by side.
- 2. Bahdanau Attention Vs Luong Attention (CampusX) — a longer walk through both papers.

Day 74 preview

- **Self-attention.** So far a decoder attends to an encoder. Day 74 lets every position of one sequence attend to every position of the same sequence, with dot-product scores, and tries it on Week 9's real tagging data.
- Today's exercise script, `day73_exercise.py`, isolates the dot and general reads, forward and backward, gradient-checked.